ECSE 554 — Final Project Report
Task Decomposition for Robotics using Small LLMs

Sabrina Yan(261153368)

April 29, 2026

1 Introduction

This project explores whether small instruction-tuned large language models (LLMs) can turn natural-language manipulation commands into structured task plans for robot execution. The target application is tabletop manipulation with a Kinova Gen3 robot in ROS2. Instead of generating free-form text, each model must produce a JSON plan that follows a fixed schema and uses only a closed set of executable robot actions. This makes the problem a constrained task-decomposition task in which both action selection and action order matter.

Following the assignment specification, the evaluation is performed on **5 validation tasks**. I compare the performance of SmolLM2-1.7B-Instruct, Qwen2.5-1.5B-Instruct, and Llama-3.2-1B-Instruct.

Each model is tested with three prompting strategies: zero-shot, one-shot, and few-shot. The goal is to identify which model-prompting configuration is most reliable for structured task decomposition, and then connect the best-performing setup to a ROS2 execution pipeline.

2 Problem Setup

2.1 Task Decomposition Format

For every instruction, the model must produce a JSON object with ordered execution steps. Each step contains an `action` and optional arguments describing manipulated objects, target objects, and spatial relations. The closed action set is:

- `move_pose`
- `open_gripper`
- `close_gripper`

This formulation is intentionally restrictive. A model is not only required to return valid JSON, but also to choose the correct primitive actions in the correct order so that the plan can be executed downstream by the robot controller.

2.2 Prompting Strategies

Three prompting strategies are compared:

- **Zero-shot:** only the task description and output requirements are provided.
- **One-shot:** the prompt includes one worked example of the desired JSON output.
- **Few-shot:** the prompt includes multiple examples of correct decompositions.

The main idea is that worked examples should reduce ambiguity, especially for small models that may otherwise struggle to follow a rigid schema and infer multi-step manipulation plans.

3 Methodology

3.1 Evaluation Protocol

Each model-prompting pair is evaluated on five validation tasks, as required by the assignment. For each task, the predicted JSON plan is compared with the corresponding ground-truth plan. Performance is measured at the *action level*: a predicted step is counted as correct only if the action matches the ground-truth action in the correct position.

The evaluation records:

- **True Positives (TP):** correctly predicted actions in the correct order.
- **False Positives (FP):** predicted actions that are incorrect, unsupported, or unnecessary.
- **False Negatives (FN):** ground-truth actions that the model failed to produce.

From these counts, precision, recall, and F1-score are computed as:

$$P = \frac{TP}{TP + FP} \tag{1}$$

$$R = \frac{TP}{TP + FN} \tag{2}$$

$$F1 = \frac{2PR}{P + R} \tag{3}$$

I also track JSON validity rate because a syntactically invalid output cannot be executed by the ROS2 parser even if some action choices are conceptually correct.

4 Results: LLM Evaluation

Table 1 reports the aggregate results averaged over the 5 validation tasks.

Model	Prompt	JSON (%)	Precision	Recall	F1
SmolLM2	Zero-shot	100.0	0.800	0.600	0.667
SmolLM2	One-shot	100.0	0.850	**0.900**	**0.867**
SmolLM2	Few-shot	100.0	**0.900**	0.750	0.810
Qwen	Zero-shot	100.0	0.597	0.500	0.517
Qwen	One-shot	100.0	0.800	0.700	0.733
Qwen	Few-shot	100.0	0.800	0.700	0.733
Llama	Zero-shot	80.0	0.233	0.250	0.231
Llama	One-shot	100.0	0.850	0.850	0.838
Llama	Few-shot	100.0	0.650	0.650	0.626

Table 1: Action-level performance across models and prompting strategies, averaged over 5 validation tasks.

The best overall result is obtained by **SmolLM2 with one-shot prompting**, which reaches the highest F1-score of 0.867 and the highest recall of 0.900. SmolLM2 with few-shot prompting achieves the highest precision of 0.900, but its lower recall reduces the final F1-score. Llama with one-shot prompting is the second-best configuration with an F1-score of 0.838, while Qwen achieves moderate but stable results under one-shot and few-shot prompting.

Zero-shot prompting is clearly the weakest setup for the required models. The largest drop appears for Llama, whose zero-shot JSON validity falls to 80% and whose action-level F1 drops to 0.231.

5 Discussion

5.1 Which Prompting Strategy Worked Best?

The new 5-task evaluation changes the earlier conclusion: **one-shot prompting is the best overall strategy**. It produces the highest F1-score for both SmolLM2 and Llama, and it matches the best performance of Qwen. This suggests that, for this task, a single high-quality example provides enough structure to teach the model the desired JSON format and decomposition pattern without overwhelming it with extra prompt context.

Few-shot prompting still helps in some cases, especially for SmolLM2 precision, but it is not universally superior. For example, SmolLM2 few-shot achieves higher precision than SmolLM2 one-shot, yet its recall is lower because it often returns incomplete plans. Llama degrades noticeably from one-shot to few-shot, which suggests that longer prompts may hurt smaller models by increasing context complexity and making step ordering harder to maintain.

Overall, the pattern is consistent with the idea that small LLMs benefit from explicit demonstrations, but too many examples can become distracting. In this project, one clear example seems to

offer the best trade-off between guidance and prompt simplicity.

5.2 Why the Best Model Performed Better

Among the tested models, SmolLM2 is the strongest overall performer. Its one-shot configuration is the most balanced, with both high recall and high precision. In practice, this means it is better at both understanding the intended manipulation task and converting that understanding into a complete action sequence.

The Hugging Face model cards help explain why this may be the case. SmolLM2-1.7B-Instruct is the largest of the three evaluated models, and its card emphasizes improvements in instruction following, reasoning, and knowledge compared with earlier SmolLM checkpoints. For a task like task decomposition, this matters because the model is not being asked to write fluent prose; it must follow a rigid schema, choose only from a closed action set, and preserve a valid execution order. A model with stronger instruction-following behavior is naturally better suited to this kind of structured output.

The model-card comparison also matches the trends in the experiments. Qwen2.5-1.5B-Instruct is presented as strong at structured outputs and JSON generation, which is consistent with its solid JSON validity and fairly stable performance in one-shot and few-shot prompting. However, its lower recall suggests that producing clean structure is not the same as producing the full correct action sequence. Llama-3.2-1B-Instruct has a much longer context window and is positioned as a lightweight instruction model for general assistant-style use, but in this project that extra context capacity did not translate into better decomposition accuracy. In fact, its performance dropped sharply in zero-shot mode and became less stable in few-shot prompting.

Overall, SmolLM2 appears superior because it offers the best balance of model size, instruction tuning, and robustness on structured generation. It does not just return valid JSON; it more often returns the right actions in the right order. That is exactly the behavior needed for moving from high-level language to executable robot plans.

5.3 Benefits and Limitations of Small LLMs for Robotics

This project highlights several benefits of small LLMs for robotic planning:

- They can translate intuitive natural-language commands into structured intermediate plans.
- They make it easy to experiment with prompting strategies and output formats.
- Their JSON outputs can be connected directly to a ROS2 executor.
- They reduce the need for a hand-written language parser for every task pattern.

However, the limitations are also clear:

- Valid JSON does not guarantee correct task semantics.

- Small models are sensitive to prompt wording and prompt length.
- Some models produce incomplete plans, which hurts recall.
- Unsupported actions or wrong action order can still appear even when the output looks well formatted.
- The models do not reason explicitly about physical feasibility, collision avoidance, or execution uncertainty.

These limitations mean that the LLM should be treated as a high-level planner rather than a trusted controller. A deterministic parser and execution layer are still necessary to validate and carry out the plan safely.

6 ROS2 Language-to-Action Pipeline

Based on the LLM evaluation, the selected configuration for integration is **SmolLM2 with one-shot prompting**. The execution pipeline is:

1. A user provides a natural-language command.
2. The LLM generates a JSON plan.
3. A ROS2 executor parses the JSON step by step.
4. Each symbolic step is mapped to one of the available robot actions.

The mapping is straightforward:

- `move_pose` maps to end-effector motion.
- `close_gripper` maps to grasping the target object.
- `open_gripper` maps to releasing the object.

This architecture cleanly separates language understanding from low-level control. The LLM proposes *what* to do, while the ROS2 stack handles *how* to do it.

7 Behavior-Level Evaluation

The assignment also requires behavior-level testing in simulation for three tasks:

- pick the cube,
- move the block to the left of the cup,

Task	Success Rate (%)
Pick the cube	100
Move the block to the left of the cup	100
Pick the cube and place it in the cup	100

Table 2: Behavior-level execution success rate in ROS2/Gazebo.

- pick the cube and place it in the cup.

All three behavior-level tests succeeded, giving a success rate of 100% for each task.

These results show that the overall pipeline is reliable for the tested tasks. Even though the LLM evaluation reveals differences in planning quality, the selected configuration is strong enough to support successful end-to-end execution in Gazebo for all required behaviors. A video demonstrating the final Gazebo execution is included with the submission.

8 Conclusion

Using the corrected 5-task evaluation protocol, the best-performing configuration is SmolLM2 with one-shot prompting. This setup achieves the highest overall F1-score and the highest recall, which makes it the most reliable option for converting natural-language commands into complete executable plans. The updated results also show that one-shot prompting works better overall than few-shot prompting for this project.

More broadly, this project demonstrates that small LLMs can serve as practical task-decomposition modules for robotics when the output space is tightly constrained and supported by a structured prompt. Their strengths lie in producing interpretable intermediate plans from natural-language instructions, while their weaknesses include sensitivity to prompt design and occasional planning errors. With a strong downstream ROS2 executor, however, these models can still form an effective language-to-action pipeline, as shown by the 100% success rate on the required behavior-level tests.